

Bit Manipulation

→ Decimal Number System

↳ Base 10 no. system

10 → (0-9)

$$342 \rightarrow 3 \times 10^2 + 4 \times 10^1 + 2 \times 10^0$$

$$\begin{matrix} 3 & 2 & 1 & 0 \\ 2 & 5 & 6 & 3 \end{matrix} \rightarrow 2 \times 10^3 + 5 \times 10^2 + 6 \times 10^1 + 3 \times 10^0$$

→ Binary Number System

↳ base - 2 system.

→ 2 (0 or 1)

$$\begin{matrix} 2 & 1 & 0 \\ 1 & 1 & 0 \end{matrix} \rightarrow 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \Rightarrow 6$$

$$\begin{matrix} 3 & 2 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{matrix} \rightarrow 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \Rightarrow 11$$

Binary to decimal conversion

$$\begin{matrix} 3 & 2 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{matrix} \rightarrow 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \Rightarrow 13$$

$$\begin{matrix} 4 & 3 & 2 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \end{matrix} \Rightarrow 21$$

$\downarrow \quad \downarrow \quad \downarrow$
 $2^4 \quad 2^3 \quad 2^0$

$$\begin{array}{cccccc} 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ \hline 1 & 0 & 1 & 1 & 0 & 1 & 0 \end{array} \rightarrow 1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 \Rightarrow 90$$

Decimal to Binary Conversion

e.g. 1) $n = 20$

2	20	
2	10	0
2	5	0
2	2	1
2	1	0
	0	1

$\begin{array}{cccc} 4 & 3 & 2 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 \end{array}$
 $\downarrow \quad \downarrow$
 $2^4 + 2^2$

e.g. 2) $n = 45 \Rightarrow (101101)_2$

2	45	
2	22	1
2	11	0
2	5	1
2	2	1
2	1	0
	0	1

Addition of Decimal Numbers

$$\begin{array}{r} \textcircled{0} \textcircled{0} \\ 368 \\ + 453 \\ \hline 821 \end{array}$$

Addition of Binary numbers

$$\begin{array}{r} \textcircled{0} \textcircled{0} \textcircled{0} \\ 10101 \\ + 1101 \\ \hline 100010 \end{array}$$

$$\begin{array}{r} \textcircled{1} \textcircled{0} \textcircled{1} \textcircled{0} \textcircled{1} \\ 01101 \\ \hline 00101 \end{array}$$

$$\begin{array}{r} \textcircled{1} \\ 110101 \\ + 100110 \\ \hline 101101 \end{array}$$

$$\begin{array}{r}
 \\
 \\
 + \\
 \hline
 \\
 \hline

 \end{array}$$

Bitwise Operators

→ used to perform operations on individual bits of binary numbers.

→ 0 → false / unset

→ 1 → True / set

And (&)

$$0 \& 0 = 0$$

$$0 \& 1 = 0$$

$$1 \& 0 = 0$$

$$1 \& 1 = 1 \quad (\text{is true only when both the i/p are true})$$

OR (|)

$$0 | 0 = 0$$

$$0 | 1 = 1 \quad (\text{is true when any of the i/p are true})$$

$$1 | 0 = 1$$

$$1 | 1 = 1$$

XOR (^)

$$0 \wedge 0 = 0$$

$$0 \wedge 1 = 1$$

$$1 \wedge 0 = 1$$

$$1 \wedge 1 = 0$$

(opposite bits generates a true)

Not (! / ~)

$$\sim 0 = 1$$

$$\sim 1 = 0$$

Example 1)

$$\begin{array}{cc} 101 & 110 \\ \uparrow & \uparrow \\ \text{print}(5 \& 6) & \Rightarrow 4 \end{array}$$

$$\begin{array}{r} 101 \\ 2 \ 110 \\ \hline 100 \end{array}$$

$$\begin{array}{r|l} 2 & 5 \\ \hline 2 & 2 \\ \hline 2 & 1 \\ \hline & 0 \end{array} \begin{array}{l} 1 \\ 0 \\ 1 \end{array}$$

Example 2

$$\begin{array}{cc} 10100 & 101101 \\ \uparrow & \uparrow \\ \text{print}(20 \& 45) & \\ \hline & \\ \downarrow & \\ 14 & \end{array}$$

$$\begin{array}{r} 010100 \\ 2 \ 101101 \\ \hline 000100 \end{array}$$

Ex-3

92 | 154



01011100

10011010

(or)

10011010

01011100

11011110 → 222

Ex4

(92 ^ 154)

→ (198)₁₀

92 ← 01011100

154 ← 10011010

11000110

Ex

20 ^ 45 → (57)₁₀



Sign bit =
4 bits

0000 → 0

1000 → 0

0001 ⇒ +1

1001 ⇒ -1

Two zeros Problem

32 → 32 bits → +ve and -ve,
32's msb weightage is -ve.

How negative numbers are stored

4(bits) $5 \rightarrow 0101$
 $-5 \rightarrow 1101$ (sign bit)

$5 \rightarrow 0000101$, $-5 \rightarrow ?$ (8bit representation)

-ve no's are stored in 2's complement form

How to find 2's complement of a no.?

- ✓ 1) Convert absolute val. the no. to binary
- ✓ 2) Invert all the bits
- ✓ 3) Add 1 to it.

(8bits)
 $-5 \rightarrow$

1)	0000101
2)	1111010
	+1
3)	1111011 $\rightarrow -5$

(8bits)
 $-3 \rightarrow$

1)	0000011
2)	1111100
	+1
3)	1111101 $\rightarrow -3$

$\begin{matrix} 3 & 2 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{matrix} \rightarrow 0$

$\begin{matrix} 3 & 2 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{matrix}$

$\rightarrow -2^3 \Rightarrow -8$

-10

1)

0 0 0 0 1 0 1 0

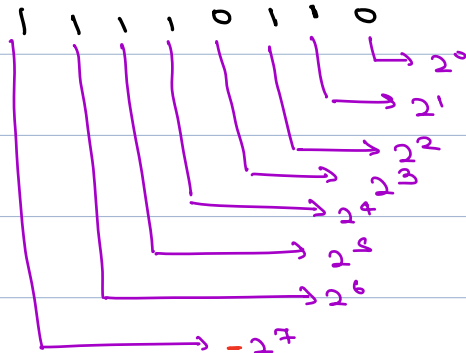
2)

1 1 1 0 1 0 1

3)

$\begin{array}{r} \\ \\ \\ \\ \\ \\ \\ \\ \hline 11110110 \end{array} \xrightarrow{+1} \underline{\underline{-10}}$

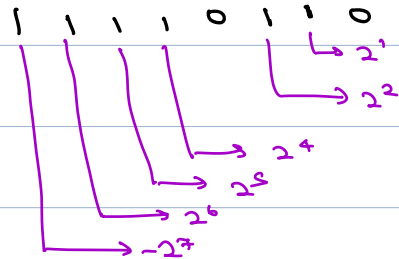
7 6 5 4 3 2 1 0



(8 bit no)

MSB weightage is -ve.

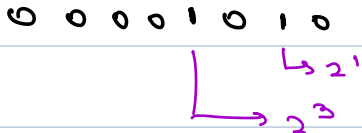
7 6 5 4 3 2 1 0



$$\Rightarrow -2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1$$

$$\Rightarrow -128 + 118 \Rightarrow -10$$

7 6 5 4 3 2 1 0



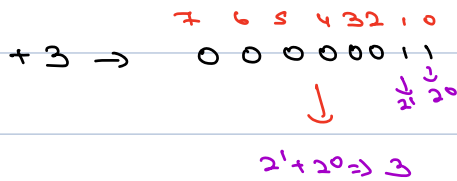
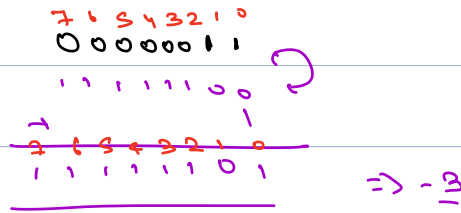
$$\Rightarrow 10$$

(8 bit)

-3 → 1)

2)

3)

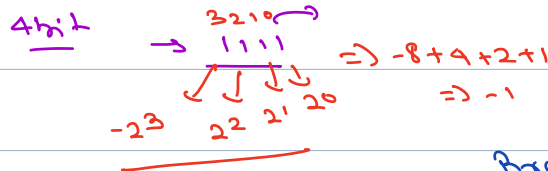


↓

$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^0$$

⇒ -3

Sign bit

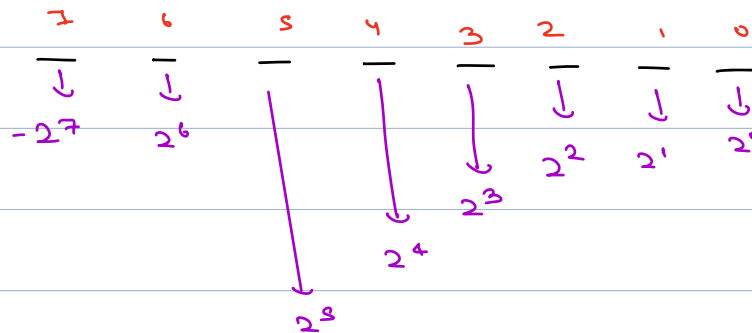


Break

10:29pm - 10:39pm

← Range of Data Types →

8 bit data type (-ve)



Biggest -ve no → 1 0 0 0 0 0 0 0

Biggest +ve no → 0 1 1 1 1 1 1 1

For n bit no

$$\begin{array}{ccccccc} \overset{n-1}{\text{---}} & \overset{n-2}{\text{---}} & & & \overset{3}{\text{---}} & \overset{2}{\text{---}} & \overset{1}{\text{---}} & \overset{0}{\text{---}} \\ \downarrow & \downarrow & & & \downarrow & \downarrow & \downarrow & \downarrow \\ -2^{n-1} & 2^{n-2} & & & 2^3 & 2^2 & 2^1 & 2^0 \end{array}$$

Biggest -ve no $\rightarrow (-2^{n-1})$ (1000....)

Biggest +ve no $\rightarrow 2^{n-2} + 2^{n-3} \dots 2^2 + 2^1 + 2^0$ (01111...)

$$\frac{a(r^n - 1)}{r - 1} \Rightarrow \frac{1(2^{n-1} - 1)}{2 - 1} \Rightarrow 2^{n-1} - 1$$

Ranges

n bit no. $(-2^{n-1} \text{ to } 2^{n-1} - 1)$

8 bit no. $(-2^7 \text{ to } 2^7 - 1) \Rightarrow (-128 \text{ to } 127)$

4 bit no. $(-2^3 \text{ to } 2^3 - 1) \Rightarrow (-8 \text{ to } 7)$

int $\rightarrow$ 32 bit no. $(-2^{31} \text{ to } 2^{31} - 1) \Rightarrow (-2 \times 10^9 \text{ to } 2 \times 10^9)$

long $\rightarrow$ 64 bit no. $(-2^{63} \text{ to } 2^{63} - 1) \Rightarrow (-8 \times 10^{18} \text{ to } 8 \times 10^{18})$

$$2^{10} \Rightarrow 1024$$

$$2^{10} \approx 1000$$

$$2^{10} \approx (10)^3$$

$$(2^{10})^3 \approx (10^3)^3$$

$$2^{30} = 10^9$$

$$2^{10} \approx 10^3$$

$$(2^{10})^6 = (10^3)^6$$

$$2^{60} = 10^{18}$$

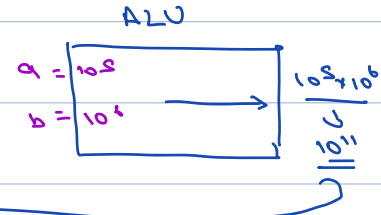
Importance of Constraints

```
int a = 105
```

```
int b = 106
```

```
int c = a * b → 1011
```

```
print c (wrong ans)
```



Arithmetic logic unit.

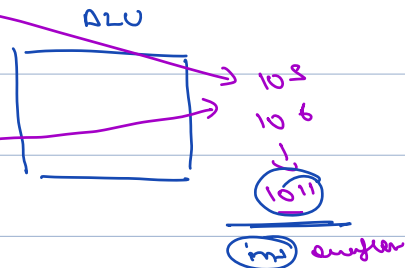
- The calculation happens at ALU.
- If we provide ALU with two INT, it calculates result in INT.
- Therefore, $a * b$ will overflow before even getting stored in c.

```
int a = 105
```

```
int b = 106
```

```
long c = a * b;
```

```
print(c)
```



Fails, the Reason:

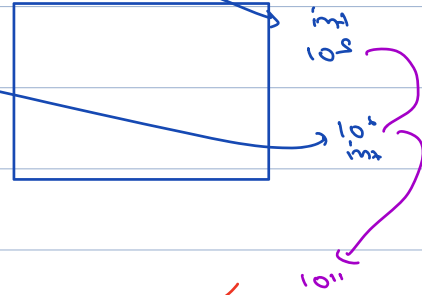
c would contain overflowed value since $a * b$ will overflow at the time of calculation, therefore there's no use to change datatype of c from INT to LONG.

```
int a = 105
```

```
int b = 106
```

```
long c = (long)a * b
```

```
Print (c)
```



Fails, the Reason:

Already overflown, hence no use to typecast later.

```
int a = 105
```

```
int b = 106
```

```
long c = (long)a * b;
```

```
Print (c)
```

10¹¹

ALL

a(long) b(int)

✓ ✓

res → long

WORKS, the Reason:

- Here, we have typecasted **a** to long before multiplying with **b**.
- If we send one INT and one LONG, ALU calculates answer in LONG.

Question

Given an array of size N, calculate the sum of array elements.

Constraints:

$1 \leq \underline{N} \leq 10^5 \rightarrow$

$1 \leq A[i] \leq \underline{10^6}$

~~long~~

```
int sum = 0;
for (int i = 0; i < N; i++) {
    sum = sum + A[i];
}
print(sum)
```

overflow

$\sim \rightarrow$

$10^6 + 10^6 + 10^6 \dots$

$\underbrace{\hspace{10em}}_{10^8}$

$\downarrow$

10^{11}